Problem Set 1 Solutions

This problem set is due **at 10:00pm ET** on **Wednesday, September 13, 2023**.

Note that problems 2(a) and 3(a) are warm-up problems. **For this week only**, they do not need to be submitted. **Starting next week**, warm-up problems need to be submitted 2 days after the pset is released.

EXERCISES (NOT TO BE TURNED IN)

Asymptotic Analysis, Recursion, and Master Theorem

- Do Exercise 4.3-2 in CLRS (4th ed.) on page 95 (similar to Exercise 4.3-7 in 3rd ed. on page 87).
- Do Exercise 4-3 in CLRS (4th ed.) on page 120 (similar to Exercise 4.3-9 in 3rd ed. on page 88).

Divide-and-conquer Algorithms

- Do Exercise 4.2-3 in CLRS (4th ed.) on page 89 (similar to Exercise 4.2-3 in 3rd ed. on page 82).
- Do Exercise 9.3-1 in CLRS (4th ed.) on page 241 (similar to Exercise 9.3-1 in 3rd ed. on page 223).

Optional Review Problem

(a) Consider the following functions. Within each group, sort the functions in asymptotically increasing order, showing strict orderings as necessary. For example, we may sort $n^3, n, 2n$ as $2n = \Theta(n) = o(n^3)$.

1. $\log\log n$, $(\log n)^3$, $\sqrt{n}$, $\log_3 n$, $\log(n^4)$.

   Solution: $\log\log n = o(\log(n^4)) = \Theta(\log_3 n) = o((\log n)^3) = o(\sqrt{n})$. Note that a change of base in the logarithm or an exponent in the logarithm is equivalent to a constant factor, so they are asymptotically equal. A change of variables can be helpful here–if you're not sure whether $(\log n)^2$ is asymptotically bigger or smaller than $O(\sqrt{n})$, replacing n with 2^m and comparing m^2 with $2^{\frac{m}{2}}$ is easier.

2. $\log(n!)$, $n^{\frac{5}{4}}$, $\sqrt[3]{n}$, $n\log n$.

   Solution: $\sqrt[3]{n} = o(\log(n!)) = \Theta(n\log n) = o\left(n^{\frac{5}{4}}\right)$. The second comparison uses Stirling's approximation for the factorial.

3. n^n, n^{n+1}, 2023^n, $e^{\log\left(n^{\log\log n}\right)}$.

   Solution: $e^{\log\left(n^{\log\log n}\right)} = o(2023^n) = o(n^n) = o(n^{n+1})$. The first comparison can be done in two steps after simplifying the exponentiation inside of the logarithm: $e^{\log n\cdot\log\log n} = o\left(2023^{\log n\cdot\log\log n}\right)$, and $\log n\cdot\log\log n = o(n)$. Note that n^n and n^{n+1} are asymptotically different because the constant is in the exponent.

(b) Let $T(n)$ be the time complexity of an algorithm to solve a problem of size n. Solve the following recurrence relations for $T(n)$.

1. $T(n) = 8T\left(\frac{n}{3}\right) + n^{\log_3 5}$

   Solution: By case 1 of the Master Theorem, since $f(n) = n^{\log_3 5}$ is polynomially less than $n^{\log_3 8}$, $T(n) = \Theta(n^{\log_3 8})$.

2. $T(n) = 4T\left(\frac{n}{2}\right) + n^2 \log^3 n$

   Solution: By case 2 of the Master Theorem, since $f(n) = n^2 \log^3 n$ is $\Theta(n^{\log_2 4} \log^k n)$ for $k = 3$, we have $T(n) = \Theta\left(n^2 \log^{(3+1)} n\right) = \Theta\left(n^2 \log^4 n\right)$.

3. $T(n) = 7T(\sqrt[3]{n}) + \log^2 n$.

   Solution: We solve this one by changing the variables. Assume that n is a power of 2, and let $n = 2^m$, then

$$T\left(2^m\right) = 7T\left(2^{\frac{m}{3}}\right) + m^2$$

   If we define $S(m) = T\left(2^m\right)$, the recurrence becomes

$$S(m) = 7S\left(\frac{m}{3}\right) + m^2$$

   We can use the Master Theorem (case 3), since $\log_3 7 < 2$ and the regularity condition holds, i.e., $7\left(\frac{m}{3}\right)^2 < m^2$. Therefore, $S(m) = \Theta(m^2)$, and $T(2^m) = \Theta(m^2)$. Changing the variable back ($m = \log n$), we have $T(n) = \Theta(\log^2 n)$.

   Alternatively we can solve this using induction. We guess that $T(n) \leq c \cdot \log^2 n$ for some $c > 0$ and prove using strong induction on n. We take for our base case that $T(n) = O(1)$ for all $n \leq 4$.

   We now set $c = 5$, assuming this is large enough to account for the $O(1)$ in the base case[1]. We will use induction to show that $T(k) \leq 5 \log^2 k$ for all $k \leq n$.

   Applying the recursive definition of $T(n)$ and then the inductive hypothesis, we have

[1] For a thorough explanation on why we chose $c = 5$ refer to the solutions to Problem 1-1.

$$
\begin{aligned}
T(k) &= 7T\left(\sqrt[3]{k}\right) + \log^2 k \\
&\leq 7 \cdot 5 \log^2 \left(\sqrt[3]{k}\right) + \log^2 k \\
&\leq 7 \cdot 5 \log^2 k \cdot \frac{1}{9} + \log^2 k \\
&\leq \left(\frac{35}{9} + 1\right) \log^2 k \\
&\leq 5 \log^2 k
\end{aligned}
$$

Thus we have proven by induction that $T(n) = O(\log^2 n)$. We can then easily see that $T(n) = \Omega(\log^2 n)$ since we have a work term of $\log^2 n$. Together this proves that $T(n) = \Theta(\log^2 n)$.

Problem 1-1. Recurrences and Asymptotics [15 points]

Solve the following recurrence, providing rigorous proof for your answer using either induction or a recursion tree. Give your answer with **Big-Theta notation**.

$$T(n) = 2T\left(\frac{n}{7}\right) + T\left(\frac{2n}{3}\right) + \Theta(n)$$

Solution:

Via recursion tree: Since the cost at each subproblem is $\Theta(m)$ (where m is the size of the subproblem), we can upper bound it with cm for some constant c. We prove by induction that the cost at the kth level of the tree is at most $\left(\frac{20}{21}\right)^{k-1} cn$. Consider the base case first: at the top level of the tree, the cost is at most cn as the original problem is of size n. Suppose, by our inductive hypothesis, that at the kth level of the tree the cost is at most $\left(\frac{20}{21}\right)^{k-1} cn$; then, at the $(k+1)$th level we have at most a cost of at most

$$2\left(\frac{20}{21}\right)^{k-1} c \cdot \frac{n}{7} + \left(\frac{20}{21}\right)^{k-1} c \cdot \frac{2n}{3} = \left(2 \cdot \frac{1}{7} + \frac{2}{3}\right)\left(\frac{20}{21}\right)^{k-1} cn = \left(\frac{20}{21}\right)^{k} cn$$

This proves our inductive step. Hence, by induction, the cost at the kth level of the tree is at most $(\frac{20}{21})^{k-1}cn$. The total cost is the sum of the costs at all levels, that is $\sum_k (\frac{20}{21})^{k-1}cn$, which is a geometric sum converging to $\Theta(n)$.

Via induction: Like in the notes for Recitation 1, we guess $T(n) \leq d \cdot n$ and prove using induction. The intuition for this guess comes from the fact that the subproblem sizes sum to $\frac{n}{7} + \frac{n}{7} + \frac{2n}{3} = \frac{20n}{21}$, which is a constant multiple of n with a coefficient less than 1.

Our base case is $T(1) \leq d \cdot 1 = d$, which is true if we choose $d \geq T(1)$.

For the inductive step, assume that for some d, $T(n) \leq d \cdot n$ for all $n < k$. Then we consider $T(k)$. Let $\Theta(n)$ be of the form $c \cdot n$ as before. Applying the recursive definition of $T(n)$ and then the inductive hypothesis, we have

$$\begin{aligned} T(k) &\leq 2T\left(\frac{k}{7}\right) + T\left(\frac{2k}{3}\right) + c \cdot k \\ &\leq 2 \cdot \left(d \cdot \frac{k}{7}\right) + d \cdot \frac{2k}{3} + c \cdot k \\ &= \left(\frac{20}{21}d + c\right) \cdot k \end{aligned}$$

If we choose d such that $\frac{20}{21}d + c \leq d$, then $T(k) \leq d \cdot k$. This occurs when $d \geq 21c$. Thus, both steps of the proof follow if we choose any $d \geq \max\{T(1), 21c\}$, and we have a valid proof by induction showing that $T(n) = O(n)$, as desired.

Note that to prove a tight asymptotic bound of $\Theta(n)$, we must prove not only $T(n) = O(n)$ but also $T(n) = \Omega(n)$. However, the latter is easier, since even without the recursive terms $T\left(\frac{n}{7}\right)$ and $T\left(\frac{2n}{3}\right)$, we have a work term of $\Theta(n)$ in our recurrence for $T(n)$, meaning that $T(n) = \Omega(n)$ and thus implying that $T(n) = \Theta(n)$, as desired.

Problem 1-2. Probability Practice: Dice Rolling [35 points]

Asami and Bolin are playing a dice game in which a pair of dice is rolled repeatedly. Asami wins if a sum of 5 is rolled before any sum greater than or equal to 8, and Bolin wins if any sum greater than or equal to 8 is rolled first. We will find the probability that Asami wins the game.

Let E_n denote the event that a 5 occurs on the nth roll and neither 5 nor any number greater than or equal to 8 occurs on any of the first $(n-1)$ rolls.

If you need to review some probability concepts, you can find the probability review handout on Canvas.

(a) *[Warm-up:* ***not to be submitted for this week****]*
What is the probability that a sum of 5 is rolled on any given roll of the pair of dice? What is the probability that any number greater than or equal to 8 is rolled on any given roll of the pair of dice?

Solution: Of the 36 equally likely possible rolls, 4 result in a total roll of 5– $(1,4)$, $(2,3)$, $(3,2)$, and $(4,1)$. Thus, the probability that a 5 is rolled on any given roll of the pair of dice is $\frac{4}{36} = \frac{1}{9}$.
We can analyze the other probability similarly. Of the 36 equally likely possible rolls, 15 result in a total roll greater than or equal to 8, with five corresponding to a total of 8, four corresponding to a total of 9, three corresponding to a total of 10, two corresponding to a total of 11, and one corresponding to a total of 12. Thus, the probability that any number greater than or equal to 8 is rolled on any given roll of the pair of dice is $\frac{15}{36} = \frac{5}{12}$.

(b) [5 points] Compute $\Pr[E_n]$.

Solution: As we found in the previous part, 5 occurs with probability $\frac{4}{36}$ and any number greater than or equal to 8 with probability $\frac{15}{36}$. Therefore, neither occurs on a given roll with probability

$$1 - \frac{4}{36} - \frac{15}{36} = \frac{17}{36}$$

Each of the n rolls required for E_n to occur are independent, so to find the probability $\Pr[E_n]$, we can multiply the probabilities that the first $(n-1)$ rolls result in neither 5 nor any number greater than or equal to 8 and the probability that the nth roll results in a 5, yielding

$$\Pr[E_n] = \left(\frac{17}{36}\right)^{n-1} \cdot \frac{1}{9}$$

(c) [10 points] Compute $\sum_{n=1}^{\infty} \Pr[E_n]$ and argue that it is the desired probability. In your argument, explain why it is correct to just sum the probabilities, because summing isn't always valid for all problems.
Hint: $\sum_{n=0}^{\infty} r^n = \frac{1}{1-r}$ *for* $|r| < 1$.

Solution: Since the events E_n are mutually disjoint and their union is precisely the event whose probability we are trying calculate, we can add their probabilities to obtain the desired result:

$$5\Pr[\text{ before } \geq 8] = \sum_{n=1}^{\infty} \Pr[E_n] = \sum_{n=1}^{\infty} \left(\frac{17}{36}\right)^{n-1} \cdot \frac{1}{9}$$

This can be evaluated using the hint in the problem to be

$$\frac{1}{1-\frac{17}{36}} \cdot \frac{1}{9} = \frac{4}{19}$$

(d) [20 points] Now, suppose that in a run of this game, the dice are rolled ten times with neither a 5 nor any number greater than or equal to 8 appearing. Given this, let X be the sum of these ten rolls. We would now like to find an upper bound for the following probability $\Pr[|X - \mathbb{E}[X]| \geq 28]$.

1. [3 points] Given that each of these ten rolls results in neither a 5 nor any number greater than or equal to 8, enumerate the possible totals for each roll and their respective probabilities.

   Solution: Because we know that we roll neither a 5 nor any number greater than or equal to 8, the only possible rolls are 2, 3, 4, 6, 7. There are $36 - 4 - 15 = 17$ rolls that result in these totals, with counts 1, 2, 3, 5, and 6, respectively. Thus, the probabilities are as follows:

Roll	Probability
2	$\frac{1}{17}$
3	$\frac{2}{17}$
4	$\frac{3}{17}$
6	$\frac{5}{17}$
7	$\frac{6}{17}$

2. [4 points] For these ten rolls, let X_i be the sum of the numbers on the face of the dice for roll i. Compute the expected value of X_i.

Solution: The expected value of X_i is given by

$$\mathbb{E}[X_i] = \sum_j j \cdot P[X_i = j] = \frac{1}{17} \cdot 2 + \frac{2}{17} \cdot 3 + \frac{3}{17} \cdot 4 + \frac{5}{17} \cdot 6 + \frac{6}{17} \cdot 7 = \frac{92}{17}$$

3. [5 points] Show that $\mathbb{E}[X] = \frac{920}{17}$, where X is the sum of these ten dice rolls.

   Solution: By linearity of expectation, we have

$$\mathbb{E}[X] = \sum_{i=1}^{10} \mathbb{E}[X_i].$$

   Thus, $\mathbb{E}[X] = 10 \cdot \frac{92}{17} = \frac{920}{17}$.

4. [5 points] Compute the variance of X, where X is again the sum of these ten dice rolls.

   Solution: Because the dice rolls are pairwise independent, we have

$$\mathrm{Var}[X] = \mathrm{Var}\left(\sum_{i=1}^{10} X_i\right) = \sum_{i=1}^{10} \mathrm{Var}[X_i]$$

   To compute the variance of a single dice roll, we use

$$\mathrm{Var}[X_i] = \mathbb{E}[X_i^2] - \mathbb{E}[X_i]^2$$

   To find $\mathbb{E}[X_i^2]$, we write

$$\mathbb{E}[X_i^2] = \sum_j j^2 \cdot \mathrm{Pr}[X_i = j] = \frac{1}{17} \cdot 2^2 + \frac{2}{17} \cdot 3^2 + \frac{3}{17} \cdot 4^2 + \frac{5}{17} \cdot 6^2 + \frac{6}{17} \cdot 7^2 = 32$$

   The variance of X_i is thus

$$\mathrm{Var}[X_i] = \mathbb{E}[X_i^2] - \mathbb{E}[X_i]^2 = 32 - \left(\frac{92}{17}\right)^2 = \frac{784}{289}$$

   It follows that the variance of X is given by

$$\mathrm{Var}[X] = 10 \cdot \frac{784}{289} = \frac{7840}{289}$$

5. [3 points] Use Chebyshev's inequality to find an upper bound for $\Pr[|X - \mathbb{E}[X]| \geq 28]$.

   Solution: Using Chebyshev's inequality and the value for the variance of X from the previous part, we find

$$\Pr\left[|X - \mathbb{E}[X]| \geq 28\right] \leq \frac{\frac{7840}{289}}{28^2} = \frac{10}{289}$$

   Note on problem 1-2: *The solution to the problem is essentially an exercise in applying technique. No real ideas or creativity are needed. Students must make sure that they understand the key steps.*

Problem 1-3. Melon Usk's Space Adventures [50 points]

Entrepreneur Melon Usk recently launched the Giraffe Heavy with record setting payload, including his flagship car, the Edison Roadster. In his ambitious future plans, he wishes to have many launch-sites scattered across the country.

Without much planning, he has chosen a set of launch-sites L, where the location of each launch-site $l_i \in L$ is represented by its Cartesian coordinates (x_i, y_i). You may assume that all the x_i and y_i are distinct.

However, in order to guarantee the viability of the coordinated launches, every launch-site must be at least 1 unit of distance away from every other launch-site as measured in Euclidean distance.

Your job as a 6.1220 student is to help Melon determine whether the set L violates this viability property. More formally, we will design an efficient algorithm to check whether there exists a pair $l_i, l_j \in L$ such that $\|l_i - l_j\| < 1$.

(a) *[Warm-up:* ***not to be submitted for this week****]*
Suppose the country is divided into a grid of squares, where each square has dimensions $\frac{1}{2} \times \frac{1}{2}$. Which of the following launch-site configurations forces you to reject L no matter the placements of the rest of the launch-sites?

- *In the same square.*
- *On the boundary of the same square.*
- *In neighboring squares (horizontally or vertically, not diagonally).*
- *More than one square apart.*

Solution: *We must reject L if two launch-sites are* **in the same square** *or* **on the boundary of the same square** *because they are within distance 1. The farthest apart they could be is at two opposite corners, which are distance $\frac{\sqrt{2}}{2} < 1$ apart. If two launch-sites are in neighboring squares or more than one square apart, they can be farther than one distance apart, although they may not be. Thus, we are not forced to automatically reject them.*

(b) [5 points] Provide a naive algorithm that determines whether there exists a pair of launch-sites less than 1 unit of distance apart. If such a pair exists, your algorithm should return any example pair, and if no pair exists, your algorithm should return FALSE.
Analyze the runtime of your solution.

Solution:
Iterate through all pairwise combinations of $l_i, l_j \in L$ and compute for each pair whether $\|l_i - l_j\| \geq 1$. Runtime is $O(n^2)$ with respect to the number of

launch-sites $n = |L|$ since we compare all possible pairs, and each comparison takes constant time to calculate the distance.

(c) [15 points] Let's begin with a somewhat contrived scenario, but it will be useful in getting to the full solution. Suppose that the country is almost one-dimensional: in particular, $-1 \leq x_i \leq 1$. The y_i can still be arbitrarily large.

Additionally suppose that you know L is not rejected by any $\frac{1}{2} \times \frac{1}{2}$ square from part (a).

Assume that the launch-sites have already been sorted in order of increasing y_i. Give and analyze an algorithm that solves Melon Usk's problem in worst-case $O(n)$ time with respect to $n = |L|$, the number of launch-sites. **Do not** include the cost of sorting in your runtime analysis.

Hint: *Solutions involving hashing (or a "dictionary") might not work! Keep in mind that hashing has a worst-case runtime of $O(n)$ per insertion, which could be too slow to be useful for this problem. Instead, can you take advantage of the very special shape of the country?*

Approaching the solution: Our solution from part (b) is slow because it checks every pair. But, a lot of these pairs feel very silly! For example, it feels very wasteful to compare many pairs of launch-sites, where one has very small y_i and the other has very large y_i. Let's try to restrict our search to launch-sites where the y_i are reasonably close.

To get started, let's think about what it looks like for a collection of launch-sites to be viable. Part (a) tells us that in any $\frac{1}{2} \times \frac{1}{2}$ square, there cannot be more than one launch-site, or else L would have been rejected. Therefore, in any 2×1 rectangle (which has 2 rows of 4 squares), we are guaranteed that there can be at most 8 launch-sites.

Solution: We present our algorithm below.

Algorithm: For each launch-site l_i in order of increasing y_i, check the distance between l_i and each of the 7 launch-sites immediately after l_i in the sort order. If any pair has distance within 1, return the pair. Else, return FALSE.

Correctness: For each launch-site l_i with coordinates (x_i, y_i), our algorithm works by trying to find another launch-site l_j such that $y_j > y_i$ and $\|l_i - l_j\| < 1$. In particular, consider the 2×1 rectangle $[-1, 1] \times [y_i, y_i + 1]$. Any such launch-site l_j must be inside this rectangle, or else its y-coordinate is too large to be close to l_i. Therefore, it's enough to only search through the launch-sites in this rectangle.

Moreover, we know this rectangle contains at most 8 launch-sites, viz., l_i and 7 others. Therefore, by searching through the 7 launch-sites immediately after l_i (and noting that the launch-sites are sorted in increasing y), we are guaranteed to have checked every launch-site inside this rectangle.

Runtime Analysis: Note that 7 is a constant. Therefore, we do a constant amount of work for each of the n launch-sites, so the overall runtime is $O(n)$.

(d) [20 points] We now return to the case where both of x_i and y_i can be arbitrarily large, and the launch-sites are not given in any sorted order. Improve your algorithm from part (b). For full credit, your algorithm should run in worst-case $O(n \log n)$ time with respect to $n = |L|$, the number of launch-sites.

Hint: *Use divide-and-conquer with part (c).*

Approaching the solution: Now, we no longer have the narrow almost-one-dimensional strip in part (c). Our motivation is to use divide-and-conquer to generate some of these narrow strips that combine to solve the full problem.

Solution: We present our algorithm below.

Algorithm: Sort the launch-sites in L by their x coordinates into a list L_x. Then:

- **Divide:** Divide L_x into two subsets of size approximately $\frac{n}{2}$, i.e., find a vertical line that splits the launch-sites in half and doesn't pass through any of the launch-sites. This produces a "left" half and a "right" half.
- **Conquer:** For each half, if it contains more than 3 launch-sites, recurse on it to determine whether either half contains a pair of launch-sites within distance 1. Otherwise, we apply a **base case**, i.e., apply the naive solution by checking all pairs of launch-sites in the set, and returning a pair within distance 1 if it exists.
- **Merge:** The conquer step has checked for pairs of points entirely in the left half and entirely in the right half. If we reach this step, it means that neither half returned any such pair. We now have to check for pairs of points where there is one in each half. To do so, we note that the only relevant points must have x-coordinate at most 1 away from the vertical dividing line; that is, they lie in a vertical strip S of width 2. Thus, we directly apply the solution from part (c) to complete the Merge step.

If none of the steps return a pair of close points, then we conclude that there exist no pairs of points within distance 1 of each other, so we return FALSE and terminate.

One important detail to be discussed is how to obtain the points in y-sorted order; this is required for the part (c) solution but we don't have it here.

- As an initial thought, we can re-sort L_x during every merge step. Unfortunately, this takes $O(n \log n)$ time per sort, which would make the Merge step too slow. Instead, below are two valid alternatives that would work.

- At the start of the algorithm, also sort the launch-sites by their y coordinates into a list L_y. Every time you recurse in the Divide step, walk through L_y and assign every point to the left or right half, keeping them in the same order. Therefore, the algorithm is provided with both L_x and L_y, which are the same set of points in two different orders.
- Make each recursive step also return the set of points sorted by y. The base case can simply sort $O(1)$ points by y-coordinate in $O(1)$ time. In the Merge case, we can perform a Merge-sort merge on the two returned lists, to obtain the y-sorted version of L_x.

Correctness: After the divide-and-conquer phase, no two launch-sites within either subset should be within distance 1 of each other *except* possibly within the strip S, which we account for in the Merge step.

The solution from part (c) also requires that L is not rejected by any $\frac{1}{2} \times \frac{1}{2}$ square. It turns out that we can't actually guarantee this about the narrow strips we generate, but, we have something similar, which turns out to be good enough. In particular, because the **Conquer** step didn't produce any pairs, neither half can contain any points that are too close. Therefore, it is guaranteed that squares *entirely within* one of the halves do not reject L. This is enough to achieve the original conclusion that searching 7 points suffices.

Runtime Analysis: Before entering the recursive divide-and-conquer, sorting the launch-sites takes $O(n \log n)$ time. At each iteration, splitting the launch-sites into two sets takes $O(n)$ time (and results in two sets each of half the size). The corresponding merge step goes through each launch-site in the strip S, and requires constant time for each launch-site. Because all the points in the merge could be in the strip, the merging step takes $O(n)$ time. We then have the following recursion for the time complexity:

$$T(n) \leq 2T\left(\frac{n}{2}\right) + O(n)$$

which solves to $O(n \log n)$ according to the Master Theorem. Adding the sorting does not change this complexity.

(e) [10 points] Describe how to modify your solution for part (d) to determine whether there are 3 launch-sites, all within distance 1 of each other. For full credit, your algorithm should run in worst-case $O(n \log n)$ time, where $n = |L|$, the number of launch-sites.

Solution: The bulk of the solution is similar to part (d). In the conquer step, for the base case, if a subset has less than 3 (or any choice of constant C) launch-sites, then check the distances among each *triplet* (as opposed to pair) in that subset.

If all triplets are valid, then the only remaining possibility is a triangle with two points on one side of the line and one point on the other. Proceed as in part (d) by finding the strip S, dividing it into squares, and checking the distance for each launch-site within some constant number of following launch-sites. Note that this time we are looking for pairs–and for each pair we also need to check the distance between them.

The extra work is still $O(n)$, so we get the same $O(n \log n)$ bound.